

ones

Create array of all ones

Syntax

```
X = ones
X = ones(n)
X = ones(sz1,...,szN)
X = ones(sz)

X = ones(__,typename)
X = ones(__,like=p)
```

Description

`X = ones` returns the scalar 1.

`X = ones(n)` returns an n-by-n matrix of ones. [example](#)

`X = ones(sz1,...,szN)` returns an sz1-by-...-by-szN array of ones where sz1,...,szN indicates the size of each dimension. For example, `ones(2,3)` returns a 2-by-3 array of ones. [example](#)

`X = ones(sz)` returns an array of ones where the size vector, sz, defines `size(X)`. For example, `ones([2,3])` returns a 2-by-3 array of ones. [example](#)

`X = ones(__,typename)` also specifies the data type (class) of X for any of the previous syntaxes. For example, `ones(5,"int8")` returns a 5-by-5 matrix of 8-bit integers. [example](#)

`X = ones(__,like=p)` specifies that X has the same data type, sparsity, and complexity (real or complex) as the numeric variable p. [example](#)

Examples

[collapse all](#)

Square Array of Ones

Create a 4-by-4 array of ones.

[Open in MATLAB Online](#)

[Copy Command](#)

```
X = ones(4)
```

[Get](#) ▼

```
X = 4x4
```

```
1     1     1     1
1     1     1     1
1     1     1     1
```

```
1    1    1    1
```

3-D Array of Ones

Create a 2-by-3-by-4 array of ones.

[Open in MATLAB Online](#)[Copy Command](#)

```
X = ones(2,3,4);  
size(X)
```

[Get](#) ▼

```
ans = 1×3
```

```
2    3    4
```

Size Defined by Existing Array

Define a 3-by-2 array A.

[Open in MATLAB Online](#)[Copy Command](#)

```
A = [1 4 ; 2 5 ; 3 6];  
sz = size(A)
```

[Get](#) ▼

```
sz = 1×2
```

```
3    2
```

Create an array of ones that is the same size as A.

```
X = ones(sz)
```

[Get](#) ▼

```
X = 3×2
```

```
1    1  
1    1  
1    1
```

Nondefault Numeric Data Type

Create a 1-by-3 vector of ones whose elements are 16-bit unsigned integers.

[Open in MATLAB Online](#)[Copy Command](#)

```
X = ones(1,3,"uint16")
```

[Get](#)

```
X = 1x3 uint16 row vector
```

```
1    1    1
```

```
class(X)
```

[Get](#)

```
ans =  
'uint16'
```

Complex One

Create a scalar 1 that is not real valued, but instead is complex like an existing array.
Define a complex vector.

[Open in MATLAB Online](#)[Copy Command](#)

```
p = [1+2i 3i];
```

[Get](#)

Create a scalar 1 that is complex like p.

```
X = ones(like=p)
```

[Get](#)

```
X =  
1.0000 + 0.0000i
```

Size and Numeric Data Type Defined by Existing Array

Define a 2-by-3 array of 8-bit unsigned integers.

[Open in MATLAB Online](#)[Copy Command](#)

```
p = uint8([1 3 5; 2 4 6]);
```

[Get](#)

Create an array of ones that is the same size and data type as p.

```
X = ones(size(p),like=p)
```

[Get](#)

```
X = 2x3 uint8 matrix
```

```
1  1  1
1  1  1
```

```
class(X)
```

 Get ▾

```
ans =
'uint8'
```

Input Arguments

[collapse all](#)

n — Size of square matrix
integer value

Size of square matrix, specified as an integer value, defines the output as a square, n-by-n matrix of ones.

- If n is 0, then X is an empty matrix.
- If n is negative, then it is treated as 0.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

sz1, ..., szN — Size of each dimension
two or more integer values

Size of each dimension, specified as two or more integer values, defines X as a sz1-by...-by-szN array.

- If the size of any dimension is 0, then X is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- If any trailing dimensions greater than 2 have a size of 1, then the output, X, does not include those dimensions.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

sz — Output size
row vector of integer values

Output size, specified as a row vector of integer values. Each element of this vector indicates the size of the corresponding dimension.

- If the size of any dimension is 0, then X is an empty array.
- If the size of any dimension is negative, then it is treated as 0.
- If any trailing dimensions greater than 2 have a size of 1, then the output, X, does not include those dimensions.

Example: `sz = [2 3 4]` defines X as a 2-by-3-by-4 array.

Data Types: double | single | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

typename — Output class

"double" (default) | "single" | "logical" | "int8" | "uint8" | ...

Output class, specified as "double", "single", "logical", "int8", "uint8", "int16", "uint16", "int32", "uint32", "int64", or "uint64".

p — Prototype
variable

Prototype, specified as a variable.

Data Types: double | single | logical | int8 | int16 | int32 | int64 | uint8 | uint16 | uint32 | uint64

Complex Number Support: Yes

Output Arguments

[collapse all](#)**X — Array of ones**

scalar | vector | matrix | multidimensional array

Array of ones, returned as a scalar, vector, matrix, or multidimensional array.

Extended Capabilities

[collapse all](#)**C/C++ Code Generation**

Generate C and C++ code using MATLAB® Coder™.

Usage notes and limitations:

- Dimensions must be real, nonnegative integers. Empty dimensions are not supported.

GPU Code Generation

Generate CUDA® code for NVIDIA® GPUs using GPU Coder™.

Usage notes and limitations:

Refer to the usage notes and limitations in the C/C++ Code Generation section. The same usage notes and limitations apply to GPU code generation.

HDL Code Generation

Generate VHDL, Verilog and SystemVerilog code for FPGA and ASIC designs using HDL Coder™.

Dimensions must be real, nonnegative integers.

Thread-Based Environment

Run code in the background using MATLAB® backgroundPool or accelerate code with Parallel Computing Toolbox™ ThreadPool.

The ones function fully supports thread-based environments. For more information, see [Run MATLAB Functions in Thread-Based Environment](#).

GPU Arrays

Accelerate code by running on a graphics processing unit (GPU) using Parallel Computing Toolbox™.

The ones function supports GPU array input with these usage notes and limitations:

- You can specify typename as 'gpuArray'. If you specify typename as 'gpuArray', the default underlying type of the array is double.

To create a GPU array with underlying type datatype, specify the underlying type as an additional argument before typename. For example, `X = ones(3,datatype,'gpuArray')` creates a 3-by-3 GPU array of ones with underlying type datatype.

You can specify the underlying type datatype as one of these options:

- 'double'
 - 'single'
 - 'logical'
 - 'int8'
 - 'uint8'
 - 'int16'
 - 'uint16'
 - 'int32'
 - 'uint32'
 - 'int64'
 - 'uint64'
- You can also specify the numeric variable p as a gpuArray.
If you specify p as a gpuArray, the underlying type of the returned array is the same as p.

For more information, see [Run MATLAB Functions on a GPU](#) (Parallel Computing Toolbox).

Distributed Arrays

Partition large arrays across the combined memory of your cluster using Parallel Computing Toolbox™.

The ones function supports distributed arrays with these usage notes and limitations:

- You can specify typename as 'codistributed' or 'distributed'. If you specify typename as 'codistributed' or 'distributed', the default underlying type of the returned array is double.

To create a distributed or codistributed array with underlying type datatype, specify the underlying type as an additional argument before typename. For example, `X = ones(3,datatype,'distributed')` creates a 3-by-3 distributed matrix of ones with underlying type datatype.

You can specify the underlying type datatype as one of these options:

- 'double'
- 'single'
- 'logical'
- 'int8'
- 'uint8'
- 'int16'

- 'uint16'
 - 'int32'
 - 'uint32'
 - 'int64'
 - 'uint64'
- You can also specify *p* as a codistributed or distributed array.
If you specify *p* as a codistributed or distributed array, the underlying type of the returned array is the same as *p*.
- For additional codistributed syntaxes, see [ones \(codistributed\)](#) (Parallel Computing Toolbox).

For more information, see [Run MATLAB Functions with Distributed Arrays](#) (Parallel Computing Toolbox).

Version History

Introduced before R2006a

See Also

[eye](#) | [zeros](#) | [rand](#) | [randn](#) | [createArray](#) | [complex](#) | [false](#) | [size](#)

Topics

[Class Support for Array-Creation Functions](#)

[Preallocation](#)

YOU MIGHT ALSO BE INTERESTED IN

[Converting Deep Learning Models Between PyTorch, TensorFlow, and MATLAB](#)

Import and export AI models between PyTorch®, TensorFlow™, and MATLAB®.

[Learn more](#)